EVALUACIÓN TÉCNICA DE PROGRAMACIÓN ORIENTADA A OBJETOS

Análisis de la arquitectura del software agroalimentario, detallando el modelado de clases y los pilares del paradigma POO aplicados en la solución:

- **Abstracción:** Se definió una estructura conceptual mediante las entidades abstractas `Producto`, `ProductoEnvasado`, `ProductoConTemperatura` y `ProductoCongelado`. El uso del modificador `abstract` impone una plantilla base obligatoria sin permitir la creación directa de objetos genéricos sin categoría definida.
- **Herencia:** La arquitectura organiza los componentes en niveles jerárquicos escalonados. Clases especializadas como `CongeladoPorAire` aprovechan directamente las propiedades de conservación de `ProductoConTemperatura` y la trazabilidad de `ProductoEnvasado`, logrando una máxima reutilización del código base.
- **Encapsulamiento:** Los datos miembros se resguardan bajo el modificador `private`. El acceso y la modificación se realizan de forma controlada mediante métodos `getter` y `setter`, lo que garantiza la integridad de los datos mediante verificaciones internas (por ejemplo, restringir la mezcla de componentes gaseosos entre 0 y 100% o validar intervalos de tiempo positivos).
- **Sobrecarga de Métodos y Constructores:** La clase aprovecha el polimorfismo estático permitiendo múltiples definiciones para un mismo identificador. Esto facilita instanciar clases con diferentes parámetros (caso de `ProductoFresco`) o convertir unidades al vuelo mediante métodos como `actualizarTemperatura` (soporte para Celsius, Kelvin y Fahrenheit) y `setTiempoExposicion` (segundos y minutos).
- **Sobrescritura de Métodos:** La firma abstracta `mostrarInformacion()` declarada originalmente en la raíz es redefinida con `@Override` en cada subnivel concreto. Mediante `super.mostrarInformacion()` se efectúa un encadenamiento de llamadas que recupera los atributos acumulados de las clases superiores y le concatena las variables particulares.
- **Polimorfismo:** La aplicación administra las distintas subclases mediante un contenedor unificado de tipo `ArrayList<Producto>`. Al iterar la lista y llamar a `mostrarInformacion()`, el entorno de ejecución de Java determina dinámicamente cuál implementación específica ejecutar según la naturaleza real de cada objeto almacenado.